



# Software Requirements Specification Document

## **ISU Class Survival Guide**

**Version 4**  
**4/ 29/2026**

ISU Survival Guide  
Ethan Harris, Julia Everett, Aaliyah Thomas,  
Lucer Szymon

Submitted in partial fulfillment of the requirements of  
IT 326 – Principles of Software Engineering

## Table of Contents

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Table of Contents                                                       | 2  |
| 1. Introduction                                                         | 3  |
| 1.1 Purpose                                                             | 4  |
| 1.2 Scope                                                               | 4  |
| 1.3 Definitions, Acronyms, and Abbreviations                            | 4  |
| 1.4 References                                                          | 4  |
| 1.5 Overview                                                            | 5  |
| 2. General Description                                                  | 5  |
| 2.1 Product Perspective                                                 | 5  |
| 2.2 User Characteristics                                                | 5  |
| 2.3 System Environment                                                  | 5  |
| 2.4 Assumptions and Dependencies                                        | 6  |
| 3. Specific Requirements                                                | 6  |
| 3.1 Functional Requirements                                             | 6  |
| 3.1.1 Sign In                                                           | 6  |
| 3.1.2 Sign Out                                                          | 7  |
| 3.1.3 Create Account                                                    | 7  |
| 3.1.4 Comment on a Course                                               | 7  |
| 3.1.5 Rate a Course                                                     | 7  |
| 3.1.6 Search for course                                                 | 8  |
| 3.1.7 Like a Comment                                                    | 8  |
| 3.1.8 Authentication account creation through two-factor authentication | 8  |
| 3.1.9 Reply to Comment                                                  | 9  |
| 3.1.10 Sort Courses by Rating                                           | 9  |
| 3.1.11 Sort Comments by Likes                                           | 10 |
| 3.1.12 Edit Comment                                                     | 10 |
| 3.1.13 Delete Comment                                                   | 10 |
| 3.1.14 Edit Account Information                                         | 11 |
| 3.1.15 Sort Courses by Difficulty                                       | 11 |
| 3.1.16 Delete Account                                                   | 12 |
| 3.1.17 Filter Courses by Level                                          | 12 |
| 3.1.18 Add Course Tags                                                  | 12 |
| 3.1.19 Add Course to Saved List                                         | 13 |

|                                                |    |
|------------------------------------------------|----|
| 3.1.20 Delete Course from Saved List           | 13 |
| 3.1.21 Toggle Anonymity                        | 13 |
| 3.1.22 Toggle notifications for course updates | 13 |
| 3.1.23 Edit course information                 | 14 |
| 3.1.24 Generate Course Rating                  | 14 |
| 3.1.25 Compare courses                         | 14 |
| 3.1.26 Create Course                           | 14 |
| 3.2 Non-Functional Requirements                | 15 |
| 3.2.1 Performance                              | 15 |
| 3.2.2 Reliability                              | 15 |
| 3.2.3 Security                                 | 15 |
| 3.2.4 Portability                              | 15 |
| 4. Design & Development                        | 15 |
| 4.1 Software Development Process               | 15 |
| 4.2 Deliverable 1                              | 15 |
| 4.2.1 Description                              | 16 |
| 4.2.2 Design                                   | 16 |
| 4.2.2.1 Course Diagram                         | 16 |
| 4.2.2.2 Activity Diagram                       | 18 |
| Figure 7: Sign In Activity Diagram             | 21 |
| 4.2.3 Development                              | 22 |
| 4.2.3.1 Description                            | 22 |
| 4.3 Deliverable 2                              | 24 |
| 4.3.1 Description                              | 24 |
| 4.3.2 Design                                   | 24 |
| 4.3.2.1 Course Diagram                         | 24 |
| 4.3.3 Development                              | 25 |
| 4.3.3.1 Description                            | 25 |
| 4.4. Deliverable 3                             | 28 |
| 4.4.1 Description                              | 28 |
| 4.4.2.1 Class Diagram                          | 28 |
| 4.4.3 Development                              | 29 |
| 4.4.3.1 Description                            | 30 |
| 4.5 Deliverable 4                              | 32 |

|                           |    |
|---------------------------|----|
| 4.5.1 Description         | 33 |
| 4.5.2.1 Class Diagram     | 33 |
| 4.5.2.2 Sequence Diagrams | 33 |

# 1. Introduction

This document aims to outline the project description in different sections. The first main section being about background knowledge and resources used to drive the project, while the second section is more about the concept of the software itself. Finally, the third section should give more depth to the functionality and requirements we want the system to incorporate.

## 1.1 Purpose

The ISU Class Survival Guide is aimed at college students new and older alike. The overall goal of the project is to create collaborative student driven software to help other students get through college and describe their experiences to help others. These experiences can be related to professors and how their courses are in terms of difficulty, and what to expect; or anything else a student finds it important to note about the college experiences they had.

## 1.2 Scope

The ISU Class Survival Guide will be a web application designed to help Illinois State University students explore and learn about academic courses. The system will use a Java backend with a SQL database to support sign-in and reliable data storage.

The scope of this project includes developing a centralized environment where students can view course information, rate courses, and leave comments on courses. The platform will also allow the users to create new courses, to ensure the information is current and reflective of student experiences. The system will not include communication features such as direct messaging, and it will not include other Universities' course information. Overall, the ISU Class Survival Guide aims to deliver an application that simplifies course exploration for Illinois State University students.

## 1.3 Definitions, Acronyms, and Abbreviations

| <i>Term</i>     | <i>Definition</i>                                                                 |
|-----------------|-----------------------------------------------------------------------------------|
| ISU             | Illinois State University                                                         |
| RateMyProfessor | A website that allows you to access information about professors at universities. |

## 1.4 References

IEEE. *IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements*

*Specifications*. IEEE Computer Society, 1998.

IBM. "Nonfunctional Requirements Checklist." IBM Cloud Architecture Center, 2024. [Online].

Available: <https://www.ibm.com/cloud/architecture/architecture/practices/nonfunctional-requirements-checklist/>

Rate My Professors, *RateMyProfessors.com*. Accessed: Jan. 26, 2026. [Online]. Available: <https://www.ratemyp Professors.com/>

Lovable, *Lovable.dev*. Accessed: Jan. 26, 2026. [Online]. Available: <https://lovable.dev/>

## 1.5 Overview

The rest of the Software Requirement specifications will go more in depth about how the system should function, such as the system environment, the dependencies we will need, and more. It will also explain what case diagram we are going to use, along with use cases themselves.

## 2. General Description

This section of the document describes the factors that go into defining our project's requirements.

### 2.1 Product Perspective

The ISU Class Survival Guide is a standalone web application designed specifically to help Illinois State University students make informed decisions when registering for courses. While existing platforms such as RateMyProfessor exist, that allow students to review professors, they focus on individual professors and do not provide feedback on the specific course sections, the workload, grading difficulty or the overall course organization.

Unlike the traditional professor rating platforms, this product looks at the overall course experience by allowing students to rate entire courses and individual sections, not just the instructors. Students can upvote helpful reviews, highlight important details such as the exam difficulty, attendance policies, and the project workout. Overall, students are able to share survival tips for succeeding in each course. This creates a student-driven knowledge base that is made specifically for ISUs curriculum and registration system. This application is independent, and self-contained but is intended to work alongside ISUs existing course registration tools by providing insights that are not available through the official university systems. By focusing on real student experiences with the course and section level, the system can fill a gap left by current platforms and help students plan schedules more efficiently.

### 2.2 User Characteristics

The primary users of the ISU Class Survival Guide are undergraduate students, graduate students, and professors at Illinois State University. Because users are not expected to have technical backgrounds, the system is designed to be simple and intuitive, allowing students to easily browse courses and submit reviews.

## 2.3 System Environment

The ISU Class Survival Guide will be a web application available on desktop devices. Written with a Java backend and a JavaScript frontend using the React library. Storage, organization, and management of user data will be supported using MySQL.

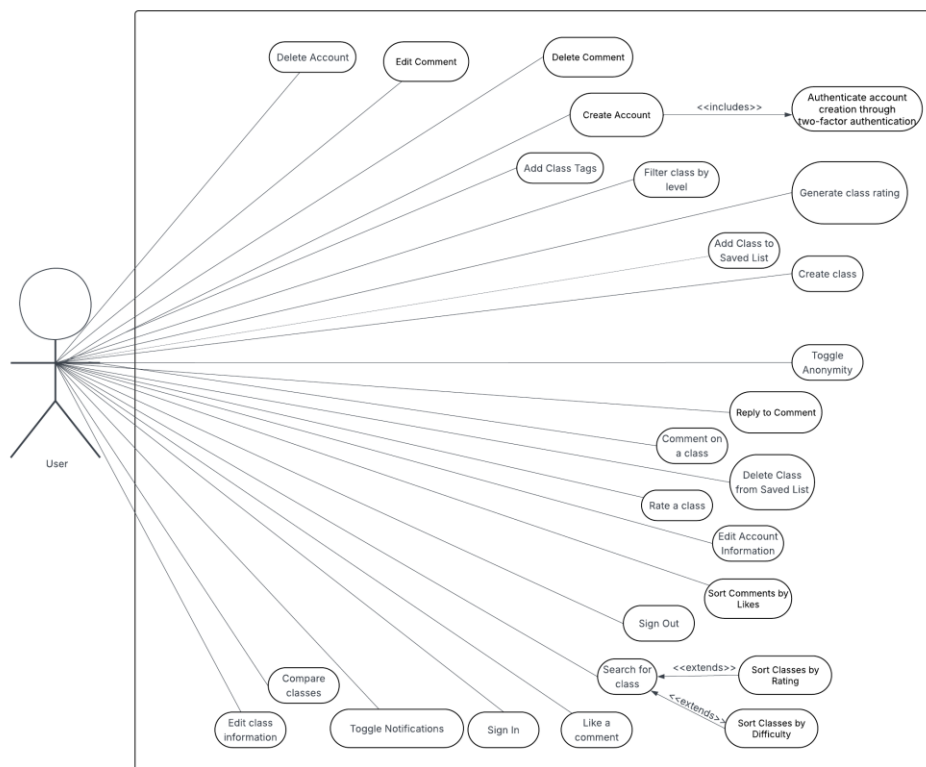
## 2.4 Assumptions and Dependencies

Our project assumes the following:

- Users are affiliated with Illinois State University, for basic authentication/access control
- Users correctly identify, label, and modify course content
- Users access the ISU Student Survival Guide using modern, up-to-date web browsers that can access the site
- For Accessibility options, users can either read the content of the website or listen to an audible option
- Users have basic experience with web applications and university systems

## 3. Specific Requirements

### 3.1 Functional Requirements



#### 3.1.1 Sign In

- Description – Allows a registered user to access the system using valid credentials.



- b. Actor(s) – User
- c. Trigger – User enters log in credentials
- d. Conditions
- e. Pre – User already has an account.
- f. Post – User is authenticated and logged in.

### **3.1.2 Sign Out**

- a. Description – Allows a logged in user to exit their account and end their session in the system.
- b. Actor(s) – User
- c. Trigger – User selects to sign out of their account.
- d. Conditions
- e. Pre – User is currently logged into the system.
- f. Post – User session is terminated and is redirected to non-signed user information.

### **3.1.3 Create Account**

- a. Description – Allows a new user to create an account in the system by providing the required information such as name, year, email address
- b. Actor(s) – User
- c. Trigger – User selects to create an account.
- d. Conditions
  - a. Pre – User does not already have an account.
  - b. Post – A new user account is created and stored in the system. User can proceed to sign in.

### **3.1.4 Comment on a Course**

- a. Description – Allows a user to post a comment on a course to share feedback or experience.
- b. Actor(s) – User
- c. Trigger – User enters a comment in the comment space
- d. Conditions
  - a. Pre – User is logged in. Selected course exists.
  - b. Post – Comment is saved and displayed under the course.

### **3.1.5 Rate a Course**

- a. Description – Allows a user to submit a 1-5 rating for a selected course that would describe the overall quality of the course. 5 being a higher score, and 1 being a lower score. User then rates the course's difficulty on the same 1-5 scale, where 5 is most difficult and 1 is least.
- b. Actor(s) – User
- c. Trigger – User selects a rating.
- d. Conditions
  - a. Pre – User is logged in. Selected course exists.
  - b. Post – Rating is saved and reflected in the course's overall rating. Note: Difficulty rating is not reflected in overall rating

### **3.1.6 Search for course**

- a. Description – Allows users to search for courses using keywords such as course name, searching a professor's name, or course number.
- b. Actor(s) – User
- c. Trigger – User enters a search term and selects to search.
- d. Conditions
  - a. Pre – System is available.
  - b. Post – Matching course results are displayed to the user.

### **3.1.7 Like a Comment**

- a. Description – Allows a user to like an existing comment on a course which will show to other users helps user determine how helpful a comment is, and helps with sorting by likes for top comments.
- b. Actor(s) – User
- c. Trigger – User likes a comment.
- d. Conditions
  - a. Pre – User is logged in. Comment exists.
  - b. Post – Comment like count is updated.

### **3.1.8 Authentication account creation through two-factor authentication**

- a. Description– When a new user creates an account and then must verify the account through two-factor authentication. After filling out the information, the system will send a verification code to the user's email and then the user must retrieve the code and enter the system to confirm ownership of the email.
- b. Actor(s) – User
- c. Trigger – Creation of a new account by a user.
- d. Conditions
  - a. Pre - User has created an account. The email being used by the new user is not already used in our system and is a functional email
  - b. Post – The user entered the correct verification code, and the system marks the email as verified. Then the account is successfully created.

### **3.1.9 Reply to Comment**

- a. Description – A user can reply to an existing comment. That new reply is now linked to the comment it is attached to.
- b. Actor(s) – User
- c. Trigger – Selecting the comment you want to reply to.
- d. Conditions
  - a. Pre - The user is signed into their account. That is a comment that the user can reply to.
  - b. Post – The reply is visible to the users.

### **3.1.10 Sort Courses by Rating**

- a. Description– When retrieving courses, the user can choose to sort the courses by highest-rated or lowest - rated.
- b. Actor(s) – User
- c. Trigger – Selecting the sort by rating when searching for the courses.
- d. Conditions
  - a. Pre – There are courses in the system and they have ratings.
  - b. Post – The system will retrieve the courses sorted by rating

### **3.1.11 Sort Comments by Likes**

- a. Description – Comments from a single course can be sorted by likes. The user can select to sort them most likely. Once received, the comments will be ordered from most likely to least liked.
- b. Actors – User
- c. Trigger – Choosing the option to sort the comments by likes when retrieving the comments for a course.
- d. Conditions
  - a. Pre – There exists a course with comments.
  - b. Post – The system retrieves the comments of a course sorted by most likes

### **3.1.12 Edit Comment**

- a. Description – How a user can edit an existing comment they have made in a course. The user will select the comment and then edit the contents. Once they are satisfied with the save, the updates.
- b. Actors – User
- c. Trigger – User wants to edit a comment.
- d. Conditions
  - a. Pre – The user is logged into their account, and they are an author of an existing comment connected to an existing course.
  - b. Post – The comment is updated in the system and is viewable by users.

### **3.1.13 Delete Comment**

- a. Description – User can delete a comment they previously posted on a course page. When deleting a comment, any replies to that comment will be deleted too.
- b. Actor – User
- c. Trigger – Selecting a comment and choosing the option to delete it.
- d. Conditions
  - a. Pre – The user is signed into their account, and there is at least one existing comment in an existing course they have created.
  - b. Post – The system removes the comment and all the replies.

### **3.1.14 Edit Account Information**

- a. Description – A user can edit their information related to their account.
- b. Actor – User
- c. Trigger – Selecting to edit account information.
- d. Conditions
  - a. Pre – The user has an existing account.
  - b. Post – The system updates the information on the account.

### **3.1.15 Sort Courses by Difficulty**

- a. Description – Filter the list of courses displayed to the user by difficulty. The user can select to sort by most difficult to least difficult or vice versa. The difficulty is taken from average among user ratings.
- b. Actor(s) – User
- c. Trigger – Select the “Sort by difficulty” option when searching for a course
- d. Conditions
  - i. Pre – User must be searching for a course
  - ii. Post – Course search results are sorted by difficulty

### **3.1.16 Delete Account**

- a. Description – Allows the user to delete their account.
- b. Actor(s) – User
- c. Trigger – User wants to delete an account.
- d. Conditions
  - i. Pre - The user is logged in.
  - ii. Post – The account and data associated with it will all be deleted.

### **3.1.17 Filter Courses by Level**

- a. Description – Filter the list of courses displayed to the user by course level. If the user only wants 100 level courses, then only 100 level courses will be shown.
- b. Actor(s) – User
- c. Trigger – Select the option to only show level 100 courses on the sorting filter
- d. Conditions
  - i. Pre - User must be accessing the entire course list and selects the sorting filter for course level
  - ii. Post – The course list is filtered by course level to show users only pertaining to that level

### **3.1.18 Add Course Tags**

- a. Description – A user can choose up to 3 course tags to assign to a course that will display quick information about the course. The information can be about finals, homework, and other information. The tags will be from a predetermined list.
- b. Actor(s) – User
- c. Trigger – User wants to assign a tag to a course.
- d. Conditions
  - i. Pre - A course must exist.

- ii. Post – A tag obtains a tag that is commonly used/voted upon among students

### **3.1.19 Add Course to Saved List**

- a. Description – If a student is in a specific course, or just wants quick access to information on a course, then they can add a course to their “list”. Which is just courses they want to come back to.
- b. Actor(s) – User
- c. Trigger – User wants to add a course to their list.
- d. Conditions
  - i. Pre - User has an account and course exists.
  - ii. Post – The course is added to their course list.

### **3.1.20 Delete Course from Saved List**

- a. Description – The course should be deleted from the users list of courses.
- b. Actor(s) – User
- c. Trigger – The user selects to delete a course from their list.
- d. Conditions
  - i. Pre - User has already added a course to their list.
  - ii. Post – The course will no longer appear in the user’s course list.

### **3.1.21 Toggle Anonymity**

- a. Description – This use case allows for a student or faculty member to post their comments anonymously, without their name being shown to other users. “Anonymous user” will be shown rather than their name.
- b. Actor(s) – User
- c. Trigger – A user wants to post anonymously.
- d. Conditions
  - a. Pre – User must have a valid account.
  - b. Post – User will have their desired anonymity setting selected and their name will no longer be visible to other users when they leave posts or comments.

### **3.1.22 Toggle notifications for course updates**

- a. Description – This use case allows for a student to allow for notifications to send to email whenever someone replies to their comment.
- b. Actor(s) – User
- c. Trigger – User selects wants to get notifications.
- d. Conditions

- a. Pre – User has an account and valid email.
- b. Post – Emails are sent whenever the someone replies to a comment they are ownership

### **3.1.23 Edit course information**

- a. Description – This use case allows for faculty users to edit course information. Non-faculty users will not be able to edit this information. Upon finishing desired changes, faculty users will save the changes made. These changes will be visible for all other users.
- b. Actor(s) – Faculty users on the site
- c. Trigger – A faculty user clicks the “Edit course” button on a given course.
- d. Conditions
  - a. Pre – Faculty user wants to change the information provided on the site for a given course.
  - b. Post – Course site is updated with the faculty user’s changes to course information.

### **3.1.24 Generate Course Rating**

- a. Description – The system aggregates stored ratings for a selected course and generates a rating distribution for the user. This allows the user to better understand the overall feedback for that course.
- b. Actor(s) – User
- c. Trigger – User requests the rating distribution for a selected course
- d. Conditions
  - a. Pre – The user is logged into the system. The selected course exists in the database.
  - b. Post – The system generates and provides the course rating distribution to the user. If no ratings exist, the system indicates that no rating data is available.

### **3.1.25 Compare courses**

- a. Description – This use case allows for a user to compare the ratings and tags between two or more courses.
- b. Actor(s) – User
- c. Trigger – User wants to compare two courses
- d. Conditions
  - a. Pre – User is signed in, and there are at least two courses to compare.
  - b. Post – Comparison of rating and tag generated.



### **3.1.26 Create Course**

- a. Description – This use case allows for a user to create a new course page, unless a course with the same name already exists
- b. Actor(s) - Student user or faculty user on the site
- c. Trigger – User selects the “Create a course” button on the home page.
- d. Conditions
  - a. Pre – User has a valid account and course must not already exist
  - b. Post – New course created or error message if course name already exists

## **3.2 Non-Functional Requirements**

### **3.2.1 Performance**

We will improve performance by adhering to programming language best practice and conducting comprehensive unit testing to ensure code efficiency, reliability, and early detection of defects.

### **3.2.2 Reliability**

We will enhance reliability by following established development standards, implementing robust error handling, and conducting thorough testing to ensure consistent system behavior and minimize failures.

### **3.2.3 Security**

We will strengthen security by adhering to secure coding best practices, implementing robust authentication and authorization mechanisms, encrypting sensitive data, and securely hashing passwords to protect user credentials from unauthorized access.

### **3.2.4 Portability**

We are using Java language which is widely used and is very portable. Any modern computer or machine with access to the internet will be able to use our site.

## **4. Design & Development**

### **4.1 Software Development Process**

The process model we have chosen for the development of our application is the Scrum Model. This is a model that will grow our application iteratively and incrementally through sprints. We liked this approach because we already have been doing a version of this. We meet every Monday at 6:30 P.M. and then assign sprints. We will start a backlog of all the tasks we need to complete in the sprints we plan to complete. During the meeting on Monday, we will have planning sessions to clarify the details of the application and make sure everyone understands their responsibilities. Since we are all students with different schedules, we modify the daily standup aspect of the Scrum model so that we check in daily at 6:30 P.M. regarding how we are all doing with our assigned work. This model will help us ensure that each sprint progresses toward our desired application.

## **4.2 Deliverable 1**

### **4.2.1 Description**

For this deliverable, our team focused on finishing the design of the application. We all created our own activity diagrams and then worked together to review our previous work and create a new course diagram. We also started testing what our development environment will look like by testing the database and creating a repository where we will share our work together.

### **4.2.2 Design**

#### **4.2.2.1 Course Diagram**

Figure 1 is our entire course diagram. This can be split into three sections. Refer to section two for database related courses. These courses are to handle the transfer of data mainly from certain courses to the database. This would be CourseHandler, AccountHandler, and CommentHandler, each allows for a median between the main courses, and the database. We have an AbstractDBHandler which also allows us to implement starting and closing connections, along with basic attributes of our database needed for execution.

Figure 3 refers to Accounts associated with course. We have AccountSettings which allows the user to toggle anonymity and change some of their account settings after they might want to. Attached to this, we have a Notifications course. This is here because we toggle notifications with account settings, which just give you an email if someone replied to one of your comments.

Finally, refer to figure 4 for course related courses. Since this is the main part of our software, and has most of our use cases, it has the most courses related to it. We have search and filter for our searching method, and filter methods, respectively. CourseTag, AddRatings, and Comment are also how a user can interact directly with a course. The commentHandler allows us to like comments and sort by likes and edit the comment.

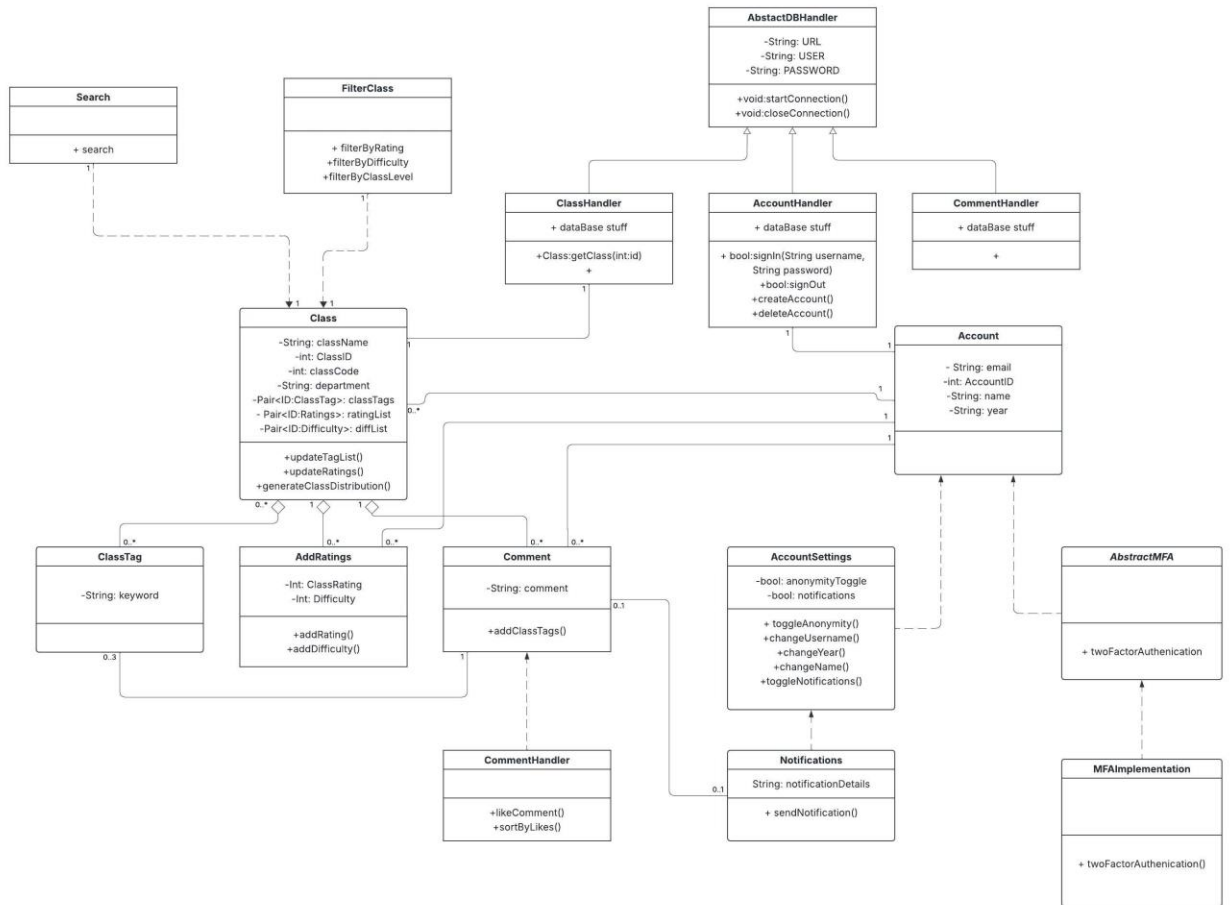


Figure 1: Entire Course Diagram

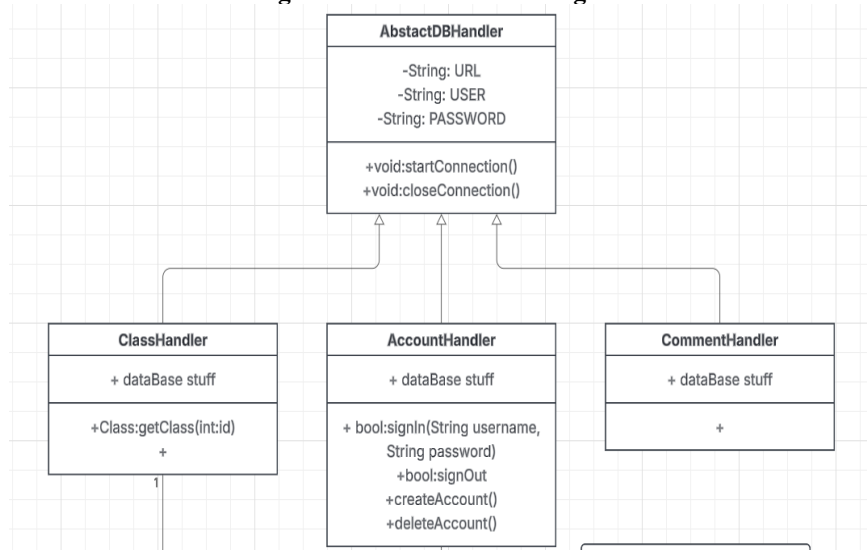


Figure 2: Database Handler Courses

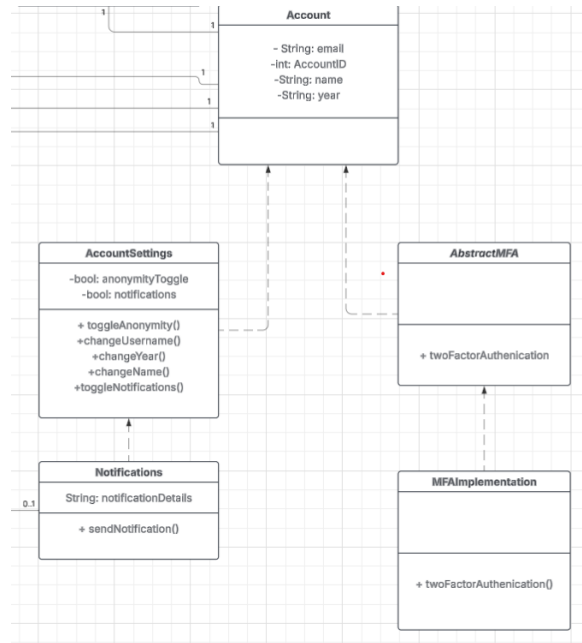


Figure 3: Account associated Courses

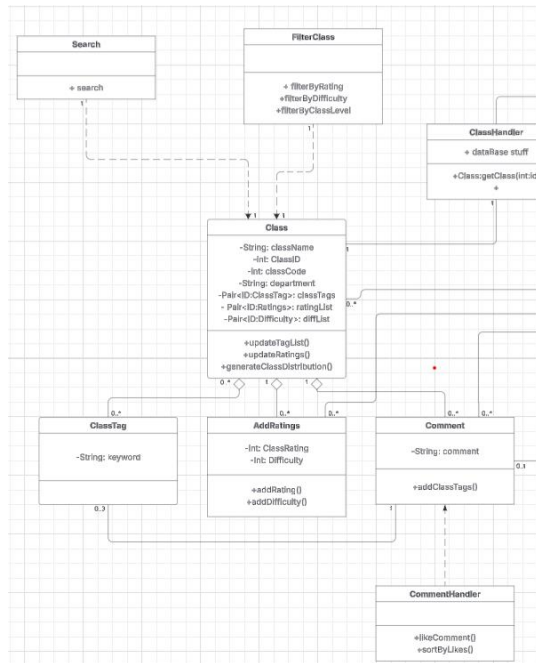
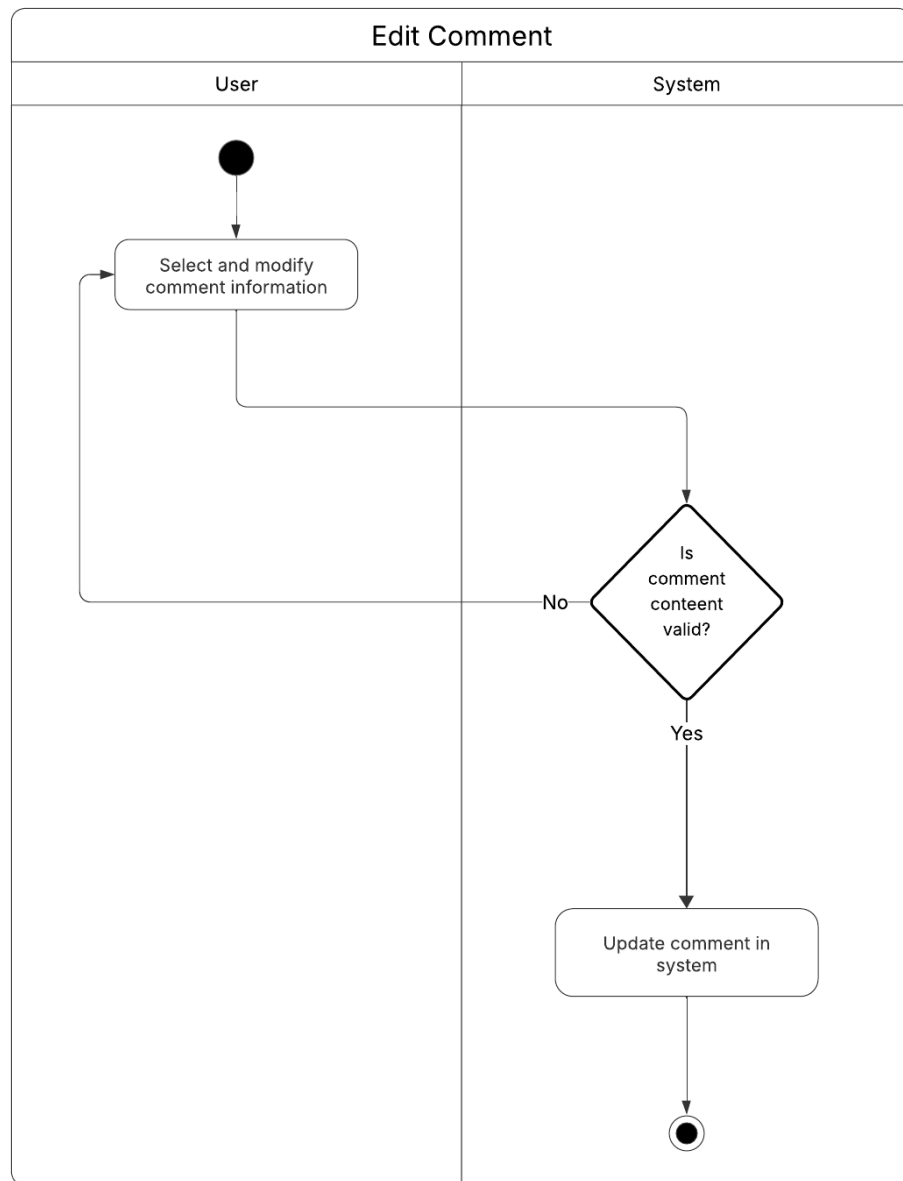


Figure 4: Course related Courses

#### 4.2.2.2 Activity Diagram

The activity diagram shown in Figure 5 explains the activities involved and their follow for editing a comment. (Use case 12: Edit Comment)



**Figure 5: Edit Comment Activity Diagram**

Figure 6 shows the use case diagram for the use case to add course tag, where a user can comment and select an option to add a course tag.

Activity Diagram - Post a Comment

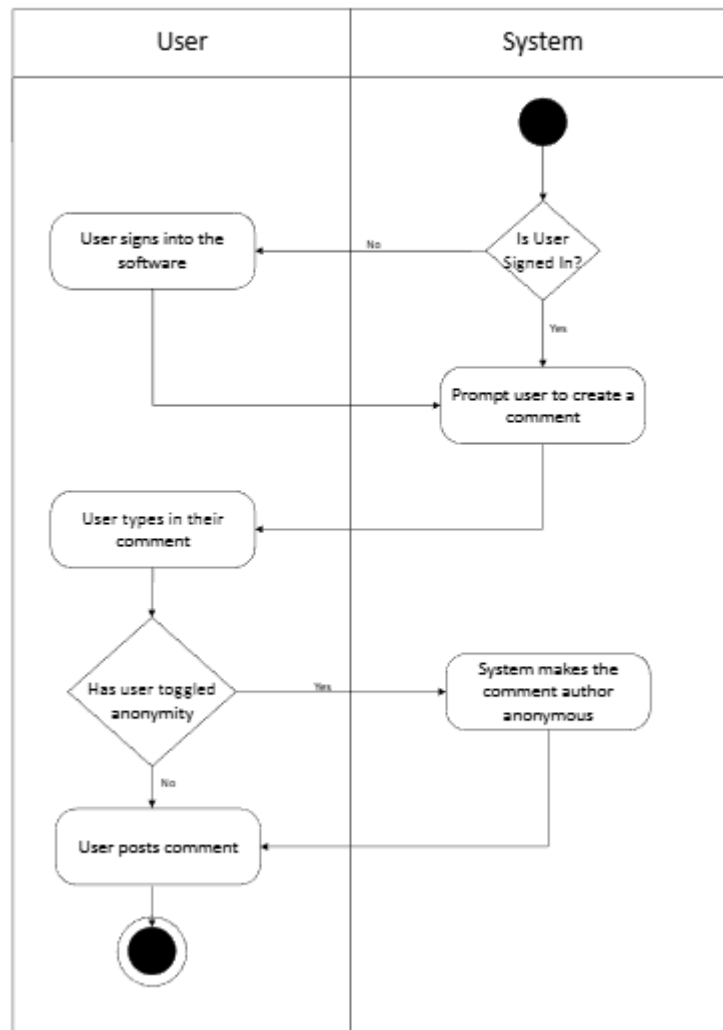
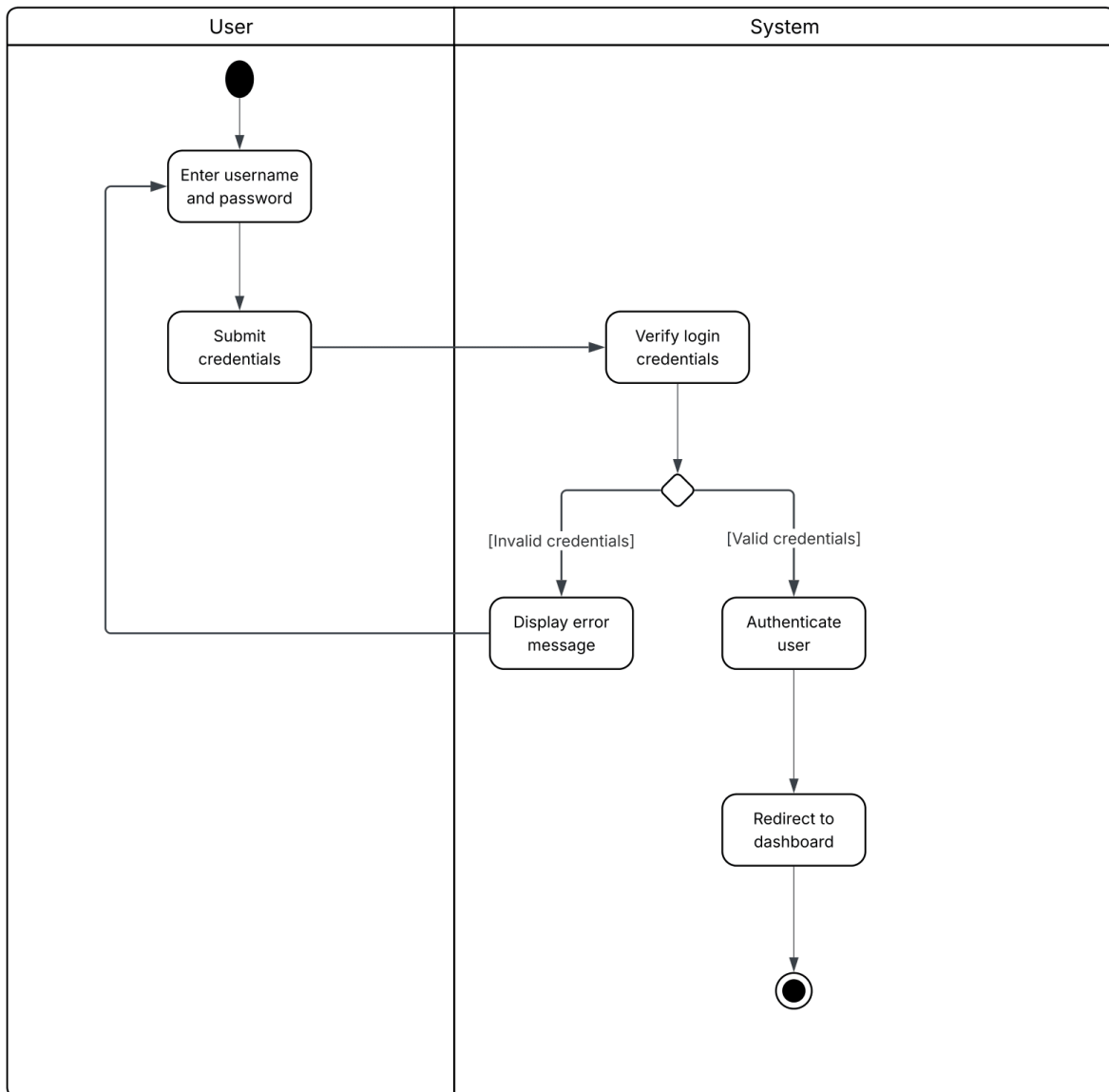


Figure 6: Post a comment Activity Diagram

The activity diagram shown in Figure 7 shows the sequence of activities involved in the sign-in process.



**Figure 7: Sign In Activity Diagram**

The activity diagram shown below in figure 8 shows the sequence of activities involved in a user rating a course.

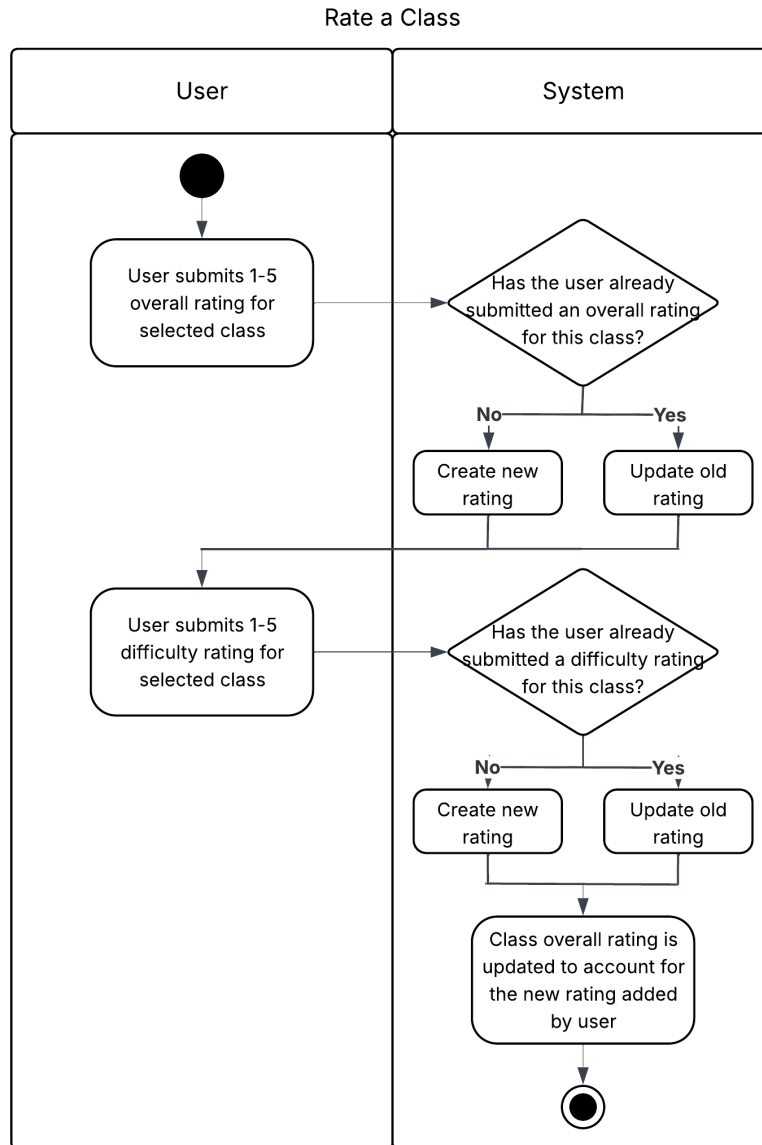


Figure 8: Rate a Course Activity Diagram

### 4.2.3 Development

#### 4.2.3.1 Description

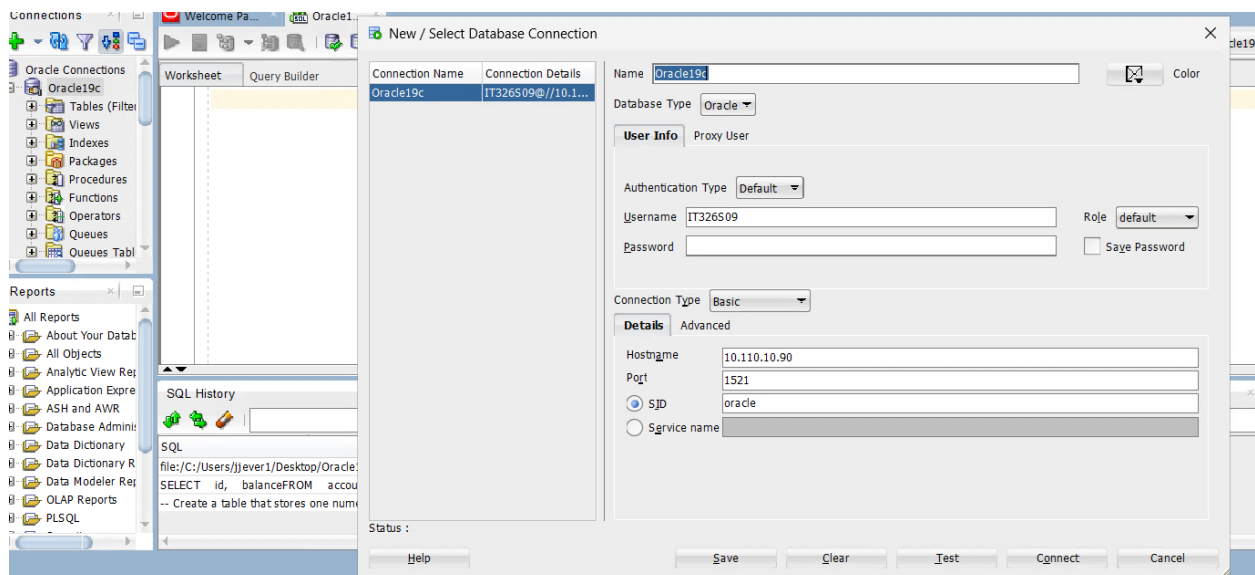
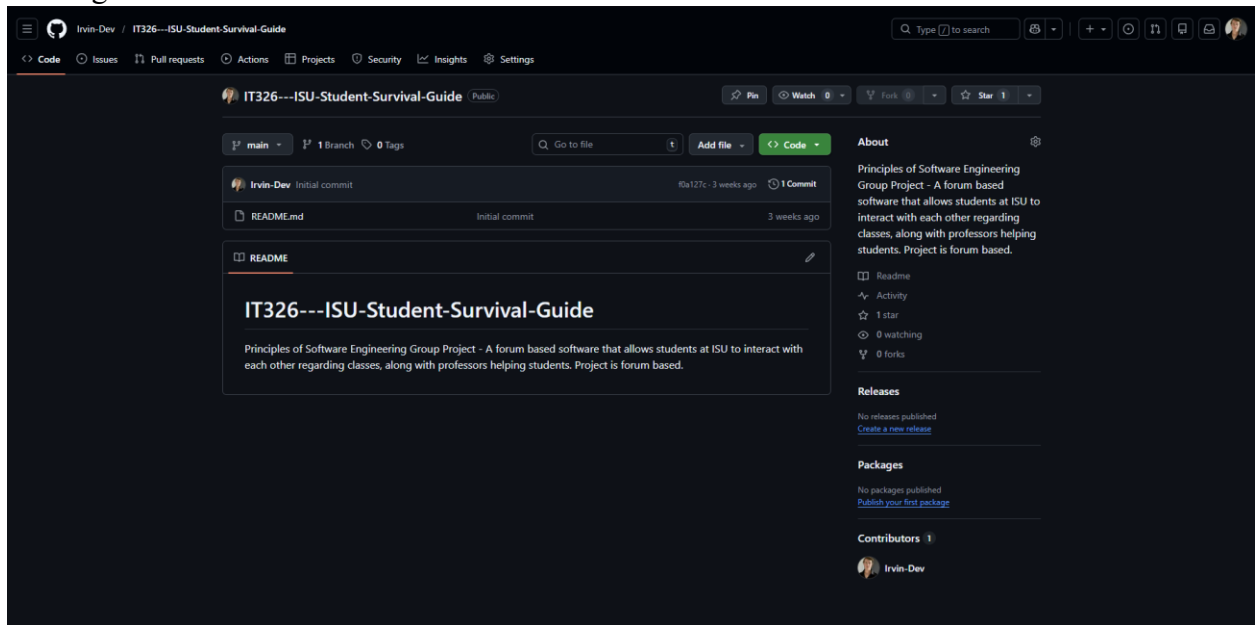
We have created the GitHub repository with this link:

Click [HERE](#) for GitHub Link



## ISU Class Survival Guide – Software Requirement Specifications

We have got the development environment working; our database has been connected. We have not started developing code full code yet. The plan is to start that next week and start implementing our course diagram.



```
PS C:\Users\jjever1\desktop> javac -cp ".;ojdbc17.jar" SQLHandler.java
PS C:\Users\jjever1\desktop> java -cp ".;ojdbc17.jar" SQLHandler
Connected to the Oracle DB!
Getting number for database: 100.0
PS C:\Users\jjever1\desktop>
```

```

Connection conn = null; // Holds the DB connection
String DB_URL = "jdbc:oracle:thin:@10.110.10.90:1521:oracle";
String USER = "IT326S09";
String PASS = "pink22";

try {
    // Connect to Oracle

    conn = DriverManager.getConnection(DB_URL, USER, PASS);

    if (conn != null) {
        System.out.println("Connected to the Oracle DB!");
    } else {
        System.out.println("Failed to make connection!");
    }

    // Query to get balance by ID
    String sql = "SELECT balance FROM account_balance WHERE id = ?";

    try (PreparedStatement state = conn.prepareStatement(sql)) {
        state.setInt(1, 1); // Set ID parameter

        try (ResultSet rs = state.executeQuery()) {
            if (rs.next()) {
                System.out.println("Getting number for database: " + rs.getDouble("balance"));
            }
        }
    }
}

```

## 4.3 Deliverable 2

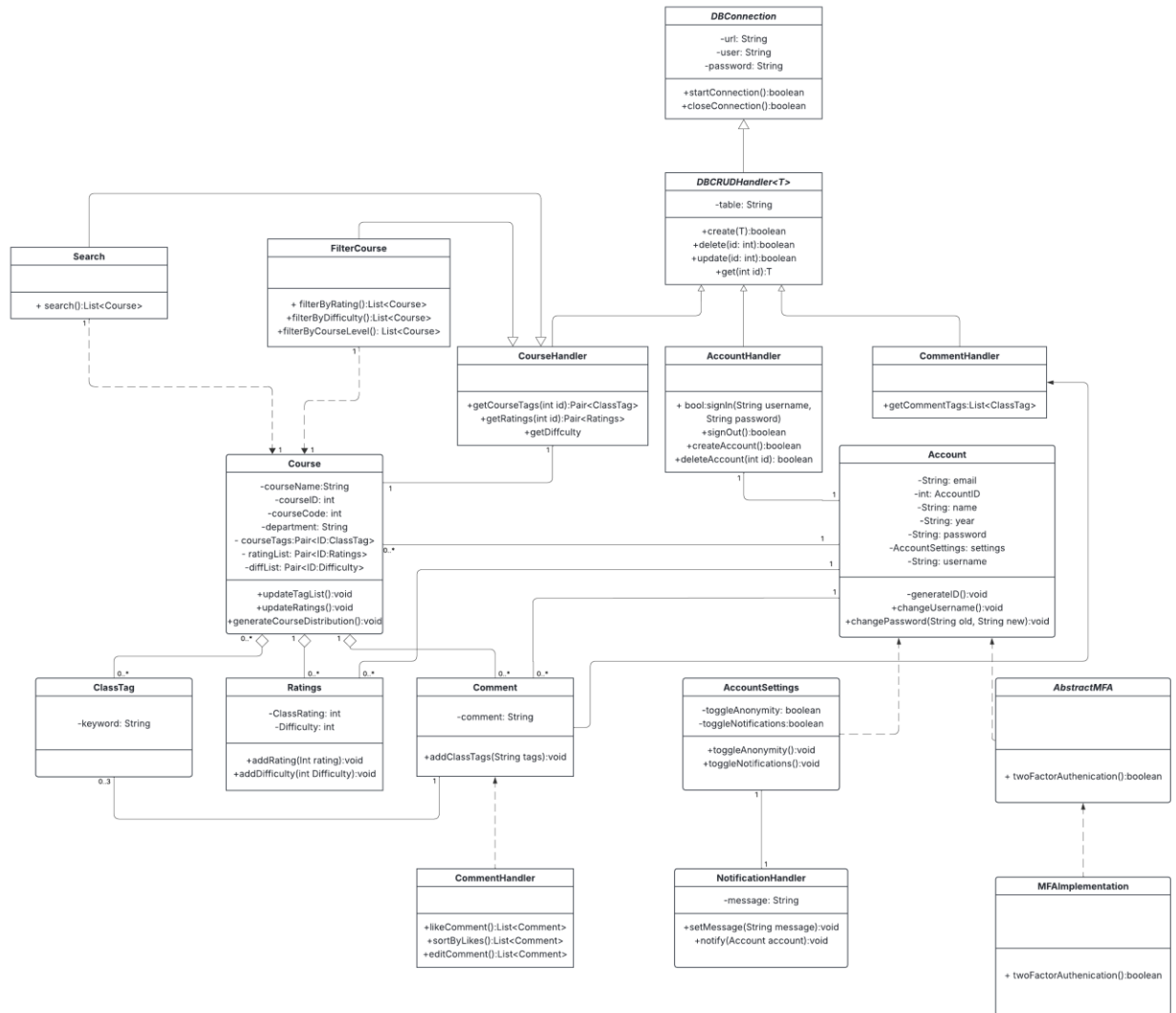
### 4.3.1 Description

We focused on fixing the problems from Deliverable 1 and starting the development of the actual application. Our team updated the course and activity diagrams based on the feedback we received. We also created course files for each course inside the course diagram. The database is also setup in Oracle SQL Developer. We now have a strong foundation to continue development and prepare us for the future.

### 4.3.2 Design

#### 4.3.2.1 Course Diagram

The image below is a picture of our class diagram.

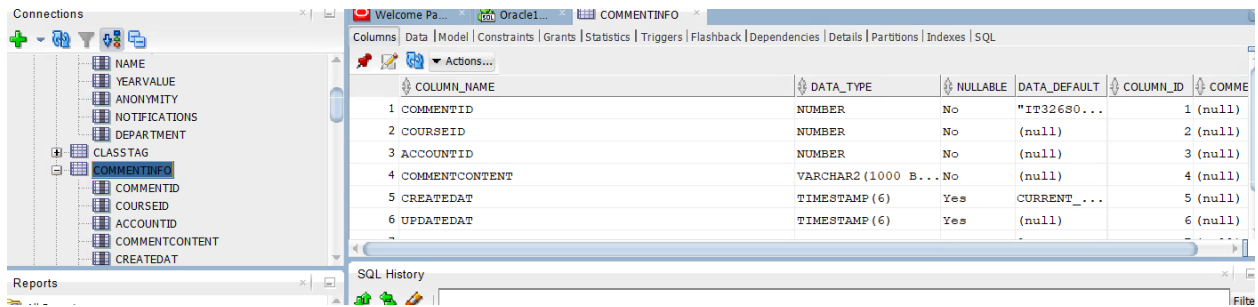


### 4.3.3 Development

#### 4.3.3.1 Description

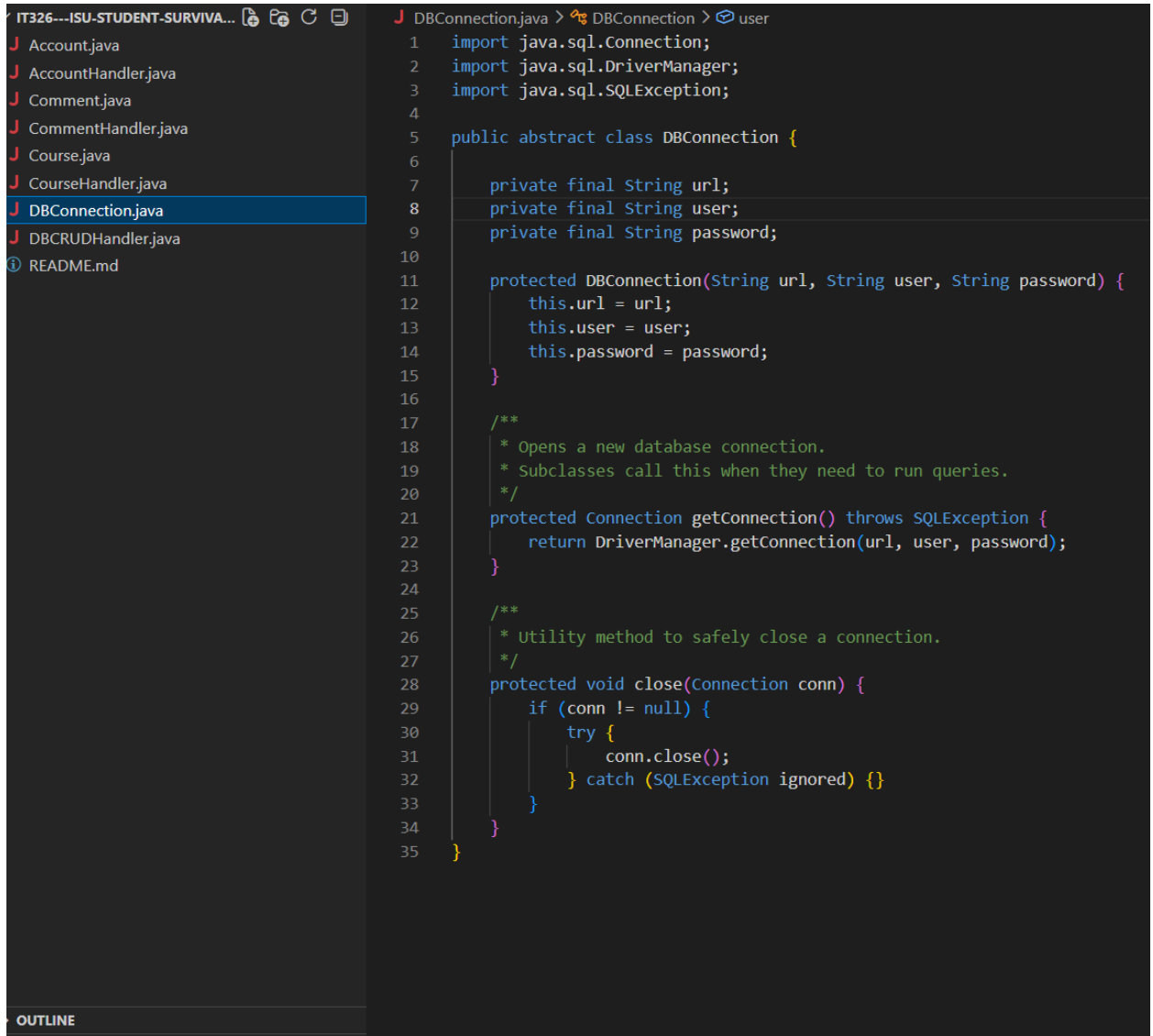
We split up the course among each team member and then created course for everything that is represented in our course diagram. Then we also worked on setting up the database. The code for the course we created is on our Github repository here: <https://github.com/Irvin-Dev/IT326---ISU-Student-Survival-Guide>. It may be split up into different branches for now.

## ISU Class Survival Guide – Software Requirement Specifications



The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connections' pane shows a tree view of the database schema, with 'COMMENTINFO' selected under the 'CLASSTAG' folder. The main window shows the 'Columns' tab for the 'COMMENTINFO' table. The table has six columns: COMMENTID, COURSEID, ACCOUNTID, COMMENTCONTENT, CREATEDAT, and UPDATEDAT. The 'SQL History' pane is visible at the bottom.

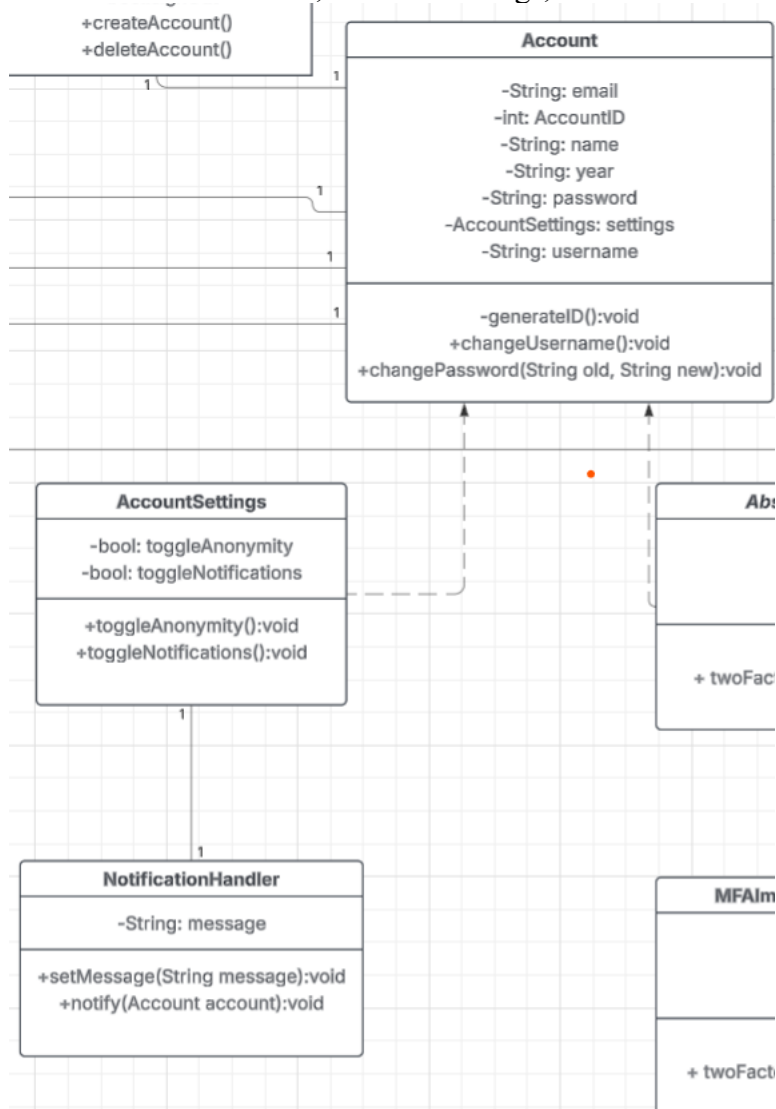
| COLUMN_ID | COLUMN_NAME    | DATA_TYPE           | NULLABLE | DATA_DEFAULT | COLUMN_ID | COMMENTS |
|-----------|----------------|---------------------|----------|--------------|-----------|----------|
| 1         | COMMENTID      | NUMBER              | No       | "IT32680..." | 1         | (null)   |
| 2         | COURSEID       | NUMBER              | No       | (null)       | 2         | (null)   |
| 3         | ACCOUNTID      | NUMBER              | No       | (null)       | 3         | (null)   |
| 4         | COMMENTCONTENT | VARCHAR2 (1000 B... | No       | (null)       | 4         | (null)   |
| 5         | CREATEDAT      | TIMESTAMP (6)       | Yes      | CURRENT_...  | 5         | (null)   |
| 6         | UPDATEDAT      | TIMESTAMP (6)       | Yes      | (null)       | 6         | (null)   |



The screenshot shows an IDE with a project named 'IT326---ISU-STUDENT-SURVIVA...'. The file explorer on the left lists several Java files, with 'DBConnection.java' selected. The main editor displays the code for 'DBConnection.java', which is an abstract class with private fields for url, user, and password, and methods for getConnection() and close().

```
DBConnection.java > DBConnection > user
1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.SQLException;
4
5 public abstract class DBConnection {
6
7     private final String url;
8     private final String user;
9     private final String password;
10
11     protected DBConnection(String url, String user, String password) {
12         this.url = url;
13         this.user = user;
14         this.password = password;
15     }
16
17     /**
18      * Opens a new database connection.
19      * Subclasses call this when they need to run queries.
20      */
21     protected Connection getConnection() throws SQLException {
22         return DriverManager.getConnection(url, user, password);
23     }
24
25     /**
26      * Utility method to safely close a connection.
27      */
28     protected void close(Connection conn) {
29         if (conn != null) {
30             try {
31                 conn.close();
32             } catch (SQLException ignored) {}
33         }
34     }
35 }
```

Ethan: Created Account, AccountSettings, and NotificationHandler course (Code is on the repo)



```

✓ public class Account {
    private String email;
    private String password;
    private int accountID;
    private String name;
    private String year;
    private AccountSettings settings;
    private String username;

    private static final Random random = new Random();

    public Account(String email, String password, String name, String year, String username) {
        this.email = email;
        this.password = password;
        this.accountID = generateID();
        this.name = name;
        this.year = year;
        this.settings = new AccountSettings();
        this.username = username;
    }

    public String getUsername() {
        return username;
    }

    public AccountSettings getSettings() {
        return settings;
    }

    public int getAccountID() {
        return accountID;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }
}

```

Szymon: Created basic classes for Course, Comment, CourseTag, and AddRatings (code in the course-classes branch on our Github repository)

Aaliyah: Created classes for AbstractMFA & MFAImplementation (code in Github)

## 4.4. Deliverable 3

### 4.4.1 Description

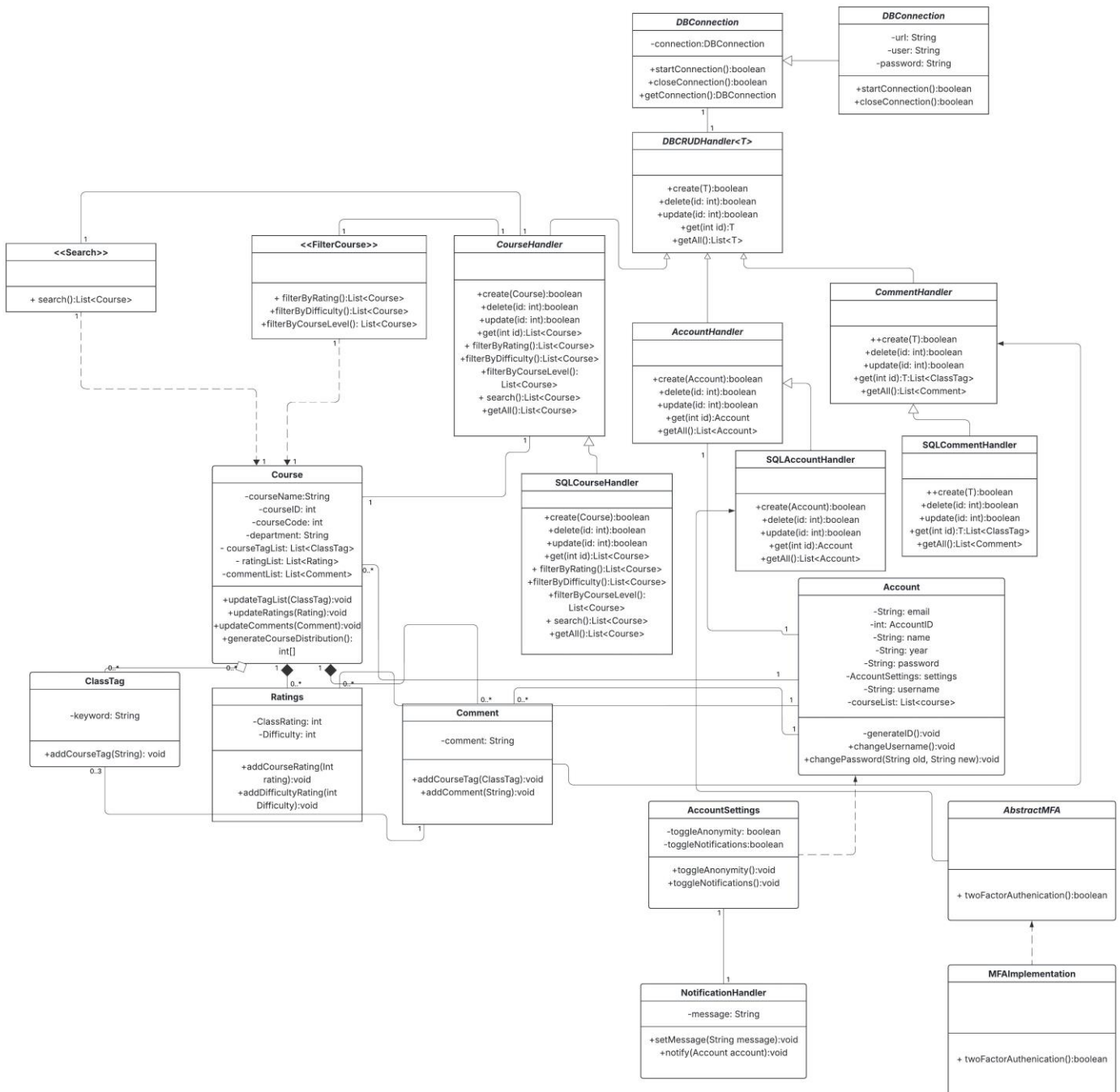
This section explains the team efforts, discussion, and work done for the “Deliverable 3”. For example, for this deliverable, we finalized our process design and started to build foundation for our software application. Some of the requirements we started to work on are the ability to login and logout of the application, and the ability of the application to be able to read in a text file.

### 4.4.2 Design

### 4.4.2.1 Class Diagram

Below is our updated class diagram:

The composition relationship between Course and Comment, and Course and Rating has been properly accounted for. Furthermore, new interfaces and handlers have been introduced to follow LSP principles our previous design violated.



### 4.4.3 Development

#### 4.4.3.1 Description

Everyone continued working on their assigned parts of the project as determined previously. Once again, here is the link to our GitHub where our additions can be found: <https://github.com/Irvin-Dev/IT326---ISU-Student-Survival-Guide>. They are scattered around in various branches.

Lucer: Worked on the course-related classes of the project. Implementations of their respective methods can now be found under the course-classes-deliverable-3 branch on the GitHub. Below is a screenshot of some of the methods implemented for the Course class specifically.



```
88  ✓ public void updateTagList(ClassTag courseTag)
89      {
90          if(courseTag == null)
91          {
92              throw new IllegalArgumentException("Course Tag can not be null.");
93          }
94          courseTagList.add(courseTag);
95      }
96
97  ✓ public void updateRatings(Rating rating)
98      {
99          if(rating == null)
100         {
101             throw new IllegalArgumentException("Rating can not be null.");
102         }
103         ratingList.add(rating);
104     }
105
106  ✓ public void updateComments(Comment comment)
107      {
108          if(comment == null)
109          {
110              throw new IllegalArgumentException("Comment can not be null.");
111          }
112          commentList.add(comment);
113      }
114
115  ✓ public int[] generateCourseDistribution()
116      {
117          // index 0 represents rating of 1, index 4 represents rating of 5
118          int[] distribution = new int[5];
119
120          if(ratingList.isEmpty())
121          {
122              System.out.println("No rating available.");
123              return distribution;
124          }
125
126          for(Rating rating : ratingList)
127          {
128              distribution[rating.getCourseRating() - 1]++;
```

Everett: Worked on implementing new designs of database handlers and testing the functionality of them.

```

Account.class      2  import java.sql.*;
Account.java       3  import java.util.ArrayList;
AccountHandler.class M 4  import java.util.List;
AccountHandler.java M 5
Comment.class      6
Comment.java       7
CommentHandler.class M 8  /**
CommentHandler.java M 9   * Handles all CRUD operations for the Comment table.
CommentHandler.java M 10  * Extends DBCRUDHandler to inherit connection handling and method structure.
CommentHandler.java M 11  */
Course.class       12
Course.java        13
CourseHandler.class M 14
CourseHandler.java 1, M 15
DBConnection.class 16
DBConnection.java  17
DBCRUDHandler.class M 18
DBCRUDHandler.java M 19
Main.class         M 20
Main.java          M 21
SQLAccountHandler... U 22
SQLAccountHandler.j... U 23
SQLCommentHandle... U 24
SQLCommentHand... 2, U 25
SQLConnection.class 26
SQLConnection.java 27
SQLCourseHandler.cl... U 28
SQLCourseHandler.ja... U 29
ojdbc17.jar        30
README.md          31

```

```

2  import java.sql.*;
3  import java.util.ArrayList;
4  import java.util.List;
5
6
7  /**
8   * Handles all CRUD operations for the Comment table.
9   * Extends DBCRUDHandler to inherit connection handling and method structure.
10  */
11  public class SQLCommentHandler extends CommentHandler {
12
13      /**
14       * Creates an CommentHandler using the given database connection.
15       *
16       * @param connection Database connection provider. */
17      public SQLCommentHandler(DBConnection connection) {
18          super(connection);
19      }
20
21      /**
22       * Inserts a new Comment into the database.
23       * Supports both top-level comments and replies (parentCommentId may be null).
24       *
25       * @param comment The Comment object to insert.
26       * @return True if the insert succeeds.
27       * @throws SQLException If the INSERT fails.
28       */
29      public boolean create(Comment comment) {
30          if (comment.getCourseId() < 0)
31              throw new IllegalArgumentException(s: "courseId must be a valid existing course.");
32
33          if (comment.getAccountId() < 0)
34              throw new IllegalArgumentException(s: "accountId must be a valid existing account.");
35
36          String sql = "INSERT INTO commentinfo "
37              + "(courseid, accountid, commentcontent, createdat, updatedat, likes, parentcommentid) "

```

Harris: Worked on generating a compatible frontend (can be found in Frontend-Testing branch) Trying to implement Spring Boot to run it. Also worked on methods for filtering and sorting trying to implement DBConnection + SQL Queries into the methods.

The screenshot shows the ISU Survival Guide website. At the top, there's a navigation bar with links like 'Courses', 'Saved', 'Compare', 'Notifications', 'Profile', 'Security', and 'Sign In'. Below the navigation bar, there's a 'Browse Courses' section with a search bar and filters. The main content area displays a grid of course cards. Each card includes the course name, a rating (e.g., 4.8, 4.6, 4.3, 4.1, 3.8, 3.5, 3.4, 3.2, 3.0), a difficulty level (e.g., 3/5, 2/5, 4/5, 4/5, 3/5, 4/5, 3/5, 5/5), and the number of reviews (e.g., 1, 2, 2, 1, 1, 1, 1, 1). The courses are categorized by department (e.g., CS, MATH, EE, PHYS, ME).

## Deliverable 3

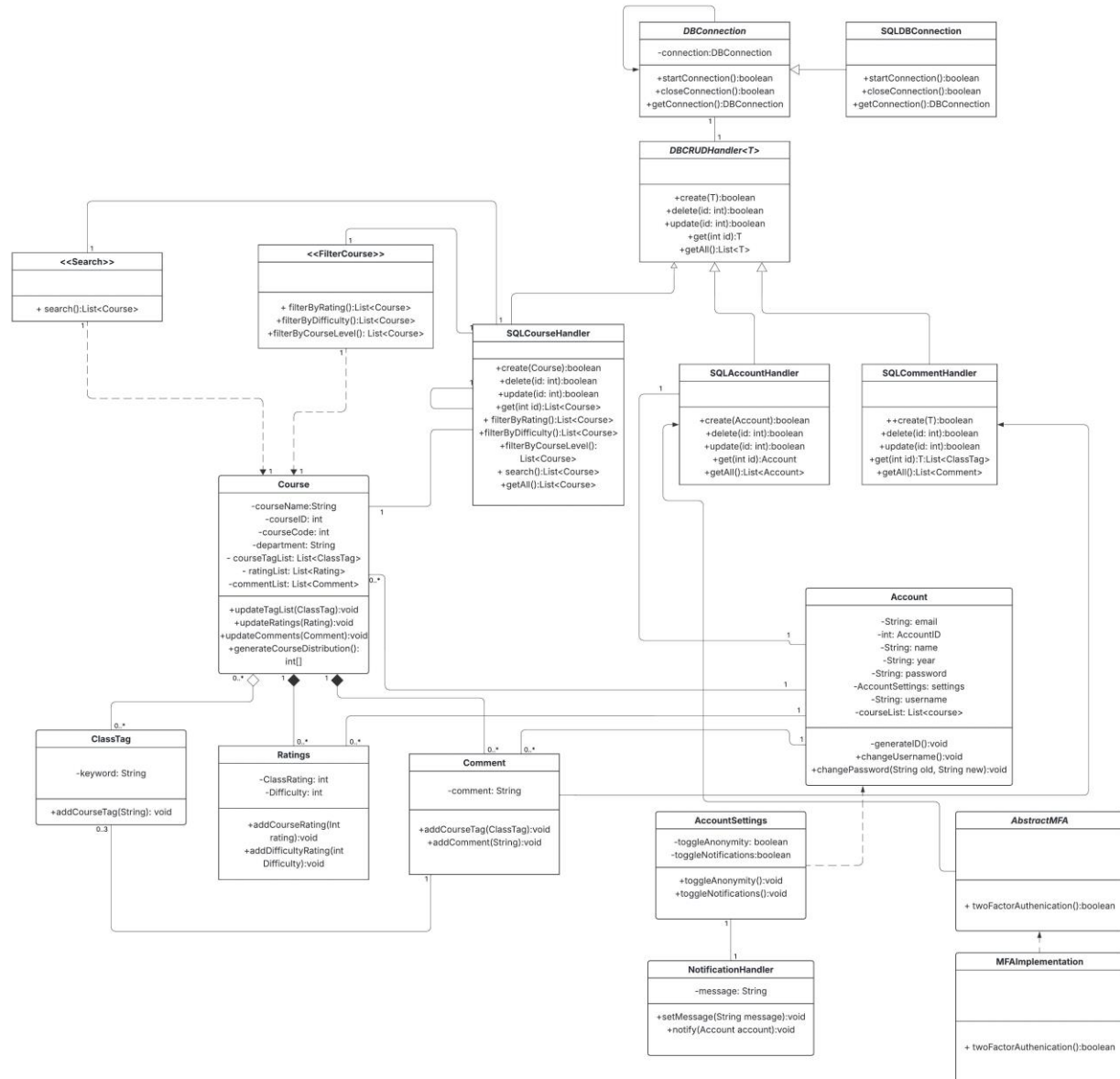
## 4.5 Deliverable 4

### 4.5.1 Description

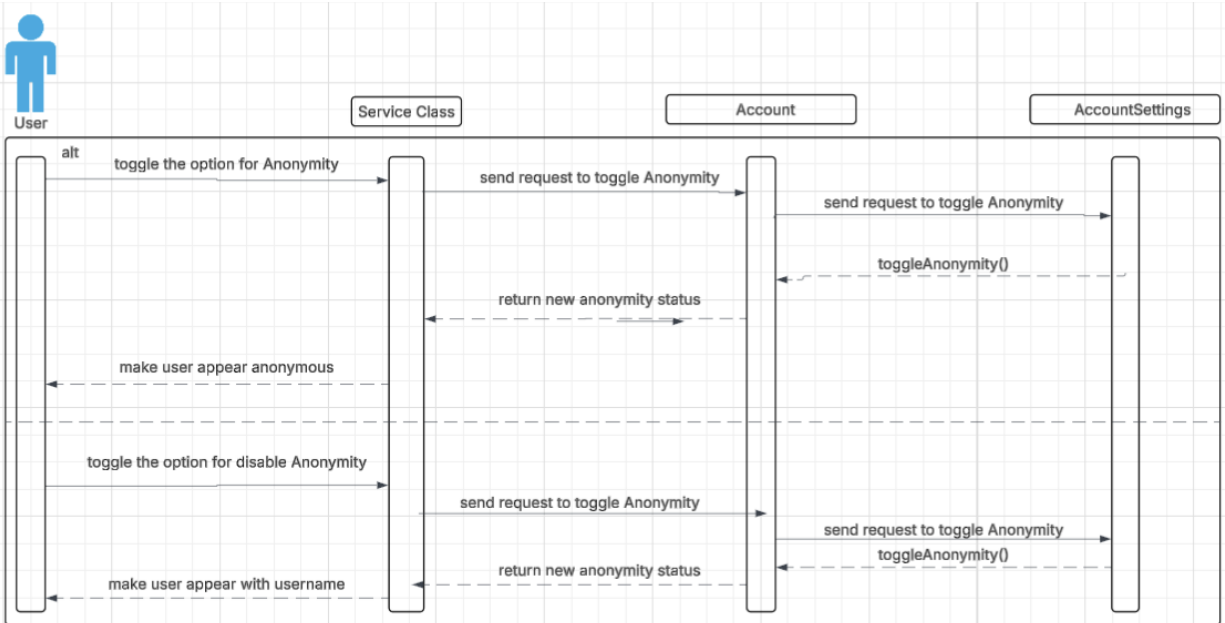
We created our sequence diagrams and updated the class diagram based on our latest design decisions. We also continued developing the application by making progress on both the frontend and backend components. Our next steps focus on deeper integration to move closer to a fully functioning web application.

### 4.5.2 Design

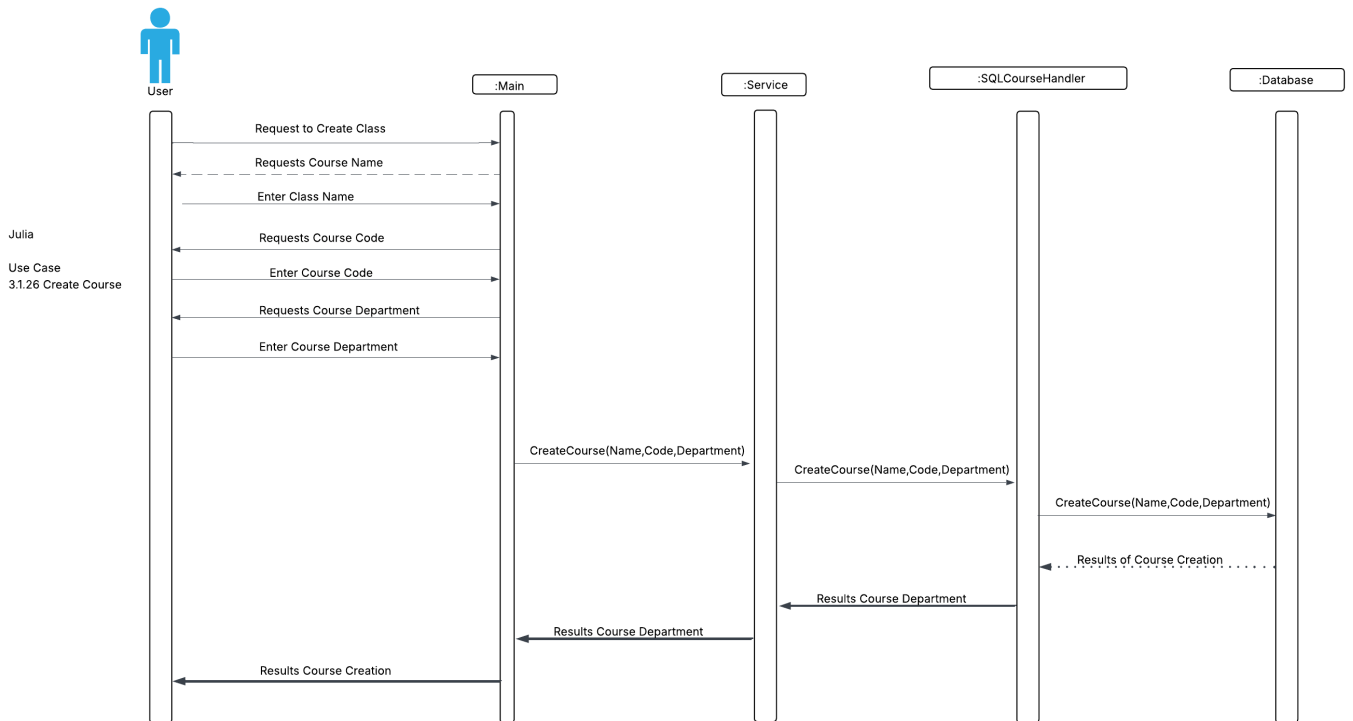
#### 4.5.2.1 Class Diagram



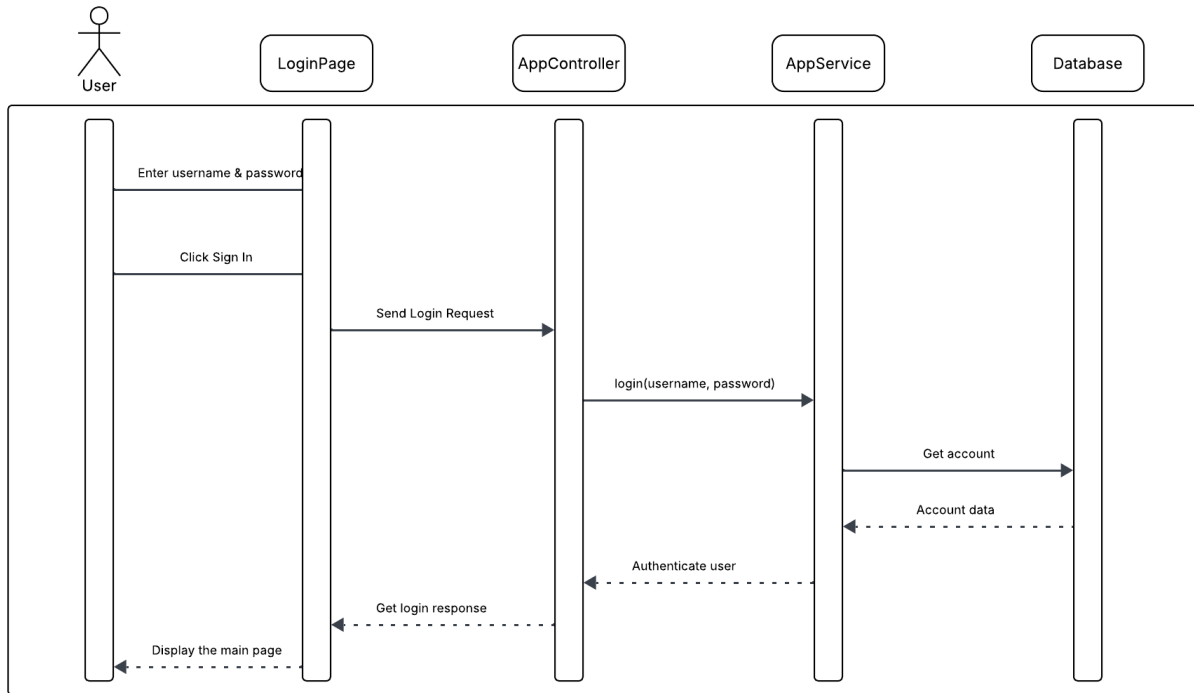
### 4.5.2.2 Sequence Diagrams



Sequence Diagram for Use Case 3.1.21 Toggle Anonymity – Ethan Harris



Sequence Diagram for Use Case 3.1.26 Create Course – Julia Everett



Sequence Diagram for Use Case 3.1.1 Sign In – Aaliyah Thomas

e